

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

EACAggregatorProxy

0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419

Ethereum

Compiler: v0.6.6+commit.6c089d02

Date: October 7, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

High

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. This is the standard Chainlink EACAggregatorProxy pattern, which is a thin, intentionally trusted proxy routing price-feed reads to an underlying aggregator selected by the contract owner. No reentrancy, delegatecall, or arithmetic-overflow exploits were found, and access-control gating via AccessControllerInterface is implemented correctly. The primary residual risk is centralization: the owner can unilaterally redirect all consumers to an arbitrary aggregator contract (or disable access control) with no timelock, multisig enforcement, or on-chain event trail for monitoring, which could facilitate price manipulation or redirection to a malicious feed if the owner key is compromised or acts maliciously.

Findings (5)

F-001

High

Centralized, unchecked control over the price-feed aggregator address

EACAggregatorProxy.sol:330–345

DESCRIPTION

``proposeAggregator`/`confirmAggregator`` (inherited from `AggregatorProxy`) allow the single ``owner`` address to redirect ``currentPhase.aggregator``, and therefore every price returned by ``latestAnswer``, ``latestRoundData``, ``getRoundData``, etc., to any arbitrary contract address, with no timelock, multisig requirement, or sanity check that the new aggregator implements a valid/consistent feed. A compromised or malicious owner key can point the proxy to a contract that reports an attacker-chosen price, instantly affecting every protocol that consumes this feed.

IMPACT

If the owner key is compromised, or the owner acts maliciously, all downstream consumers of this price feed (lending markets, derivatives, liquidation engines, etc.) can be fed arbitrary price data, leading to mass undercollateralized borrowing, wrongful liquidations, or direct fund drains in dependent protocols.

RECOMMENDATION

Require aggregator proposals to pass through a timelock (e.g., 24-48h delay) before ``confirmAggregator`` can finalize them, and/or require owner to be a multisig/DAO governance contract rather than an EOA. Emit a dedicated event (e.g., ``AggregatorUpdated(address indexed previous, address indexed current)``) in ``setAggregator`` so off-chain monitors can detect and react to changes before they take effect.

```
function confirmAggregator(address _aggregator) external onlyOwner() {
    require(_aggregator == address(proposedAggregator), "Invalid proposed aggregator");
    delete proposedAggregator;
    setAggregator(_aggregator);
}
```

F-002

Medium

Owner can disable or redirect access control via setController with no validation or event

EACAggregatorProxy.sol:471–475

DESCRIPTION

`setController` lets the owner set accessController` to any address, including address(0)` , which fully disables the checkAccess` gating on all price-reading functions (per the address(ac) == address(0)` bypass in the modifier). There is no event emitted and no zero-address guard, so this privileged state change is silent on-chain.`

IMPACT

An owner (or a compromised owner key) can silently open previously gated price data to any caller, or point access control at a malicious controller that arbitrarily grants/denies access, without any auditable event trail for monitoring systems.

RECOMMENDATION

Emit an event (e.g., `AccessControllerUpdated(address indexed previous, address indexed current)` on every change, and consider requiring explicit confirmation/timelock for disabling access control entirely.`

```
function setController(address _accessController) public onlyOwner() {
    accessController = AccessControllerInterface(_accessController);
}
```

F-003

Low

Missing zero-address validation on owner-controlled setters

EACAggregatorProxy.sol:330–339

DESCRIPTION

`proposeAggregator` , setAggregator` (via confirmAggregator`/constructor), and setController` do not validate that the supplied address is non-zero or a contract. Setting the aggregator to address(0)` or an EOA will cause all proxy read functions to revert until corrected, effectively bricking the feed.`

IMPACT

Operational misconfiguration (fat-fingered address, deployment script bug) can cause a denial-of-service of the price feed with no on-chain guard to prevent it.

RECOMMENDATION

Add `require(_aggregator != address(0), "zero address");` (and optionally an extcodesize/interface check) in setAggregator` and setController` before committing the new address.`

```
function setAggregator(address _aggregator) internal {
    uint16 id = currentPhase.id + 1;
    currentPhase = Phase(id, AggregatorV2V3Interface(_aggregator));
    phaseAggregators[id] = AggregatorV2V3Interface(_aggregator);
}
```

F-004 • SWC-101

Informational

Unchecked arithmetic on phase id increment (theoretical overflow)

EACAggregatorProxy.sol:334–335

DESCRIPTION

Compiled under Solidity 0.6.6 (pre-0.8 checked arithmetic), `uint16 id = currentPhase.id + 1;` will silently wrap to 0 once currentPhase.id` reaches 65535. This would require 65,535 aggregator changes to trigger and is not practically reachable, but is technically an unchecked overflow.`

IMPACT

Negligible in practice given the astronomically high number of aggregator rotations required to trigger wraparound, but would cause phase-id collisions in `phaseAggregators` mapping if ever reached.

RECOMMENDATION

Use SafeMath (already available in the Chainlink ecosystem for 0.6.x) or add an explicit `require(currentPhase.id < type(uint16).max)` guard for defense-in-depth, though this is low priority.

```
function setAggregator(address _aggregator) internal {
    uint16 id = currentPhase.id + 1;
    ...
}
```

F-005

Low

No event emitted on aggregator/phase changes

EACAggregatorProxy.sol:334–339

DESCRIPTION

`setAggregator` (called from the constructor and `confirmAggregator`) updates `currentPhase` and `phaseAggregators` but emits no event, unlike `transferOwnership`/`acceptOwnership` which emit `OwnershipTransferRequested`/`OwnershipTransferred`.

IMPACT

Off-chain indexers, monitoring dashboards, and automated risk systems cannot reliably detect aggregator rotations by listening for events; they must poll `aggregator()`/`phaseId()` instead, delaying detection of a malicious or erroneous feed switch.

RECOMMENDATION

Add `event AggregatorUpdated(uint16 indexed phaseId, address indexed aggregator);` and emit it inside `setAggregator`.

```
function setAggregator(address _aggregator) internal {
    uint16 id = currentPhase.id + 1;
    currentPhase = Phase(id, AggregatorV2V3Interface(_aggregator));
    phaseAggregators[id] = AggregatorV2V3Interface(_aggregator);
    // missing: emit AggregatorUpdated(id, _aggregator);
}
```

Gas Optimizations

Cache `currentPhase.aggregator` in deprecated getters to avoid repeated SLOADs

EACAggregatorProxy.sol:148–170

`latestAnswer()` and `latestTimestamp()` each perform a storage read via `currentPhase.aggregator` directly; consider caching `Phase` memory `phase = currentPhase;` once per call as already done in `latestRound()`/`latestRoundData()` for consistency and minor gas savings on repeated field access.

Redundant struct copy to memory not needed for single-field access

EACAggregatorProxy.sol:243–244

In `latestRound()`, copying the full `Phase` struct to memory just to read two fields is fine but could be simplified to two direct storage reads if gas golfing is desired; negligible impact, informational only.

Appendix - Metadata

Contract	EACAggregatorProxy
Address	0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419
Chain	Ethereum
Compiler	v0.6.6+commit.6c089d02
Audit ID	portfolio-chainlink-eth-usd
Date	10/7/2026

Files analyzed

[user] EACAggregatorProxy.sol

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-chainlink-eth-usd)

<https://cryptohawking.com/audit/manual?ref=portfolio-chainlink-eth-usd>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>